

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Analytical Problem Solving
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	ABDUL WAHID. H	Reg. No.:	192321079
Section / Batch:	B.TECH IT	Date:	03/08/2026

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyse in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyse how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct

I Abdul Wahid. H / 192321079 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ABDUL WAHID. H

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

Q1. Signed 8-bit Addition and Subtraction

Given A=+45, B=-23

$$+45 = 00101101$$

$$+23 = 00010111$$

$$1\text{'s complement} = 11101000 \text{ 2's}$$

$$\text{complement} = 11101001 \text{ (-23)}$$

Addition:

$$00101101$$

$$+11101001$$

$$100010110$$

Ignore carry => 00010110 = 22 Overflow: No

(operands have different signs) Subtraction:

$$45 - (-23) = 45 + 23$$

$$00101101$$

$$+00010111$$

$$01000100 = 68$$

Overflow: No.

Explanation:

- Positive numbers are stored directly in binary.
- Negative numbers are represented using 2's complement.
- Overflow occurs only when two numbers of the same sign produce a result of opposite sign.
Since the results are within -128 to +127, no overflow occurs.

Q2. Carry Look-Ahead Adder

Carry Look-Ahead Adder

Example: A=1011, B=1101, C0=0

$$P=A \oplus B = 0110$$

$$G=A \cdot B = 1001$$

$$C1=G0+P0C0$$

$$C2=G1+P1G0+P1P0C0$$

$$C3=G2+P2G1+P2P1G0+P2P1P0C0$$

$$C4=G3+P3G2+P3P2G1+P3P2P1G0+P3P2P1P0C0$$

Term	Formula	Purpose
Propagate	$P=A \oplus B$	Propagates carry
Generate	$G=A \cdot B$	Generates carry
C1	$G0+P0C0$	Carry 1
C2	$G1+P1G0+P1P0C0$	Carry 2

Explanation:

- CLA calculates all carry bits simultaneously.
- Ripple Carry Adder waits for each carry sequentially.
- Hence CLA has much lower propagation delay and is used in high-speed processors.

Q3. Booth Multiplication

Booth's Multiplication

Multiply $-6 \times +5$

$M = -6 = 11111010$

$Q = 00000101$

$A = 00000000$ $Q-1 = 0$

Apply Booth rules for 8 cycles:

01 $\rightarrow A = A + M$

10 $\rightarrow A = A - M$

00/11 $\rightarrow$ No operation

Arithmetic right shift after each cycle.

Final Product = $11100010 = -30$

Cycle	Q0Q-1	Operation	Shift	Remark
1	10	$A = A - M$	ARS	Subtract
2	01	$A = A + M$	ARS	Add
3-8	00/11	No Operation	ARS	Continue

Explanation:

- Booth's algorithm efficiently multiplies signed numbers.
- It reduces unnecessary additions for consecutive 1's, improving speed.

Q4. Restoring and Non-Restoring Division

Restoring Division Example: 13/3

Dividend=1101

Divisor=0011

Shift left, subtract divisor.

If result negative, restore remainder and set quotient bit=0.

Else quotient bit=1.

Final Quotient=0100 (4) Remainder=0001 (1)

Explanation:

- Restoring division restores the previous remainder whenever subtraction becomes negative.
- Non-restoring division skips this restoration during each iteration, making it faster and requiring fewer operations.

Q5. IEEE-754 Floating Point

IEEE-754 Representation of -13.25

$$13.25 = 1101.01$$

$$\text{Normalized} = 1.10101 \times 2^3$$

Single Precision:

$$\text{Sign} = 1$$

$$\text{Exponent} = 3 + 127 = 130 = 10000010 \quad \text{Mantissa} = 101010000000000000000000$$

32-bit:

$$1 \ 10000010 \ 101010000000000000000000$$

Double Precision:

$$\text{Exponent} = 3 + 1023 = 1026 = 10000000010$$

$$\text{Mantissa} = 1010100$$

Field	Single Precision	Meaning
Sign	1	Negative
Exponent	10000010	Bias 127
Mantissa	101010000000000000000000	Fraction

Explanation:

- IEEE-754 stores a floating-point number using three fields: Sign, Exponent, and Mantissa.
- The exponent is stored using a bias (127 for single precision, 1023 for double precision).
- Normalization improves precision and ensures a unique representation.